

InsightFlow

Human Activity Recognition using Deep Learning

Project Report

Image & Video Activity Classification with MobileNetV2 Transfer Learning

SUBMITTED BY

Team Member	Roll Number
Matham Amolika	HUB2603A073
Paridhi Prasanna	HUB2603A088
Yarlagadda Mahathi	HUB2603A121

Table of Contents

Abstract 3

Introduction 3

Dataset 3

Methodology..... 4

Implementation..... 5

Results and Evaluation 6

Conclusion 6

Future Scope 7

Abstract

InsightFlow is a deep learning system designed to recognize human activities from images and video using a convolutional neural network built with TensorFlow and Keras. The system classifies 15 everyday human activities — including calling, clapping, cycling, dancing, drinking, eating, fighting, hugging, laughing, listening to music, running, sitting, sleeping, texting, and using a laptop — from RGB visual input of size $224 \times 224 \times 3$.

The project implements a complete machine learning pipeline covering dataset preparation, model design, training with transfer learning, quantitative evaluation, and inference on both static images and video clips. A MobileNetV2-based transfer learning architecture was selected for its balance between accuracy and computational efficiency, making the system suitable for near real-time activity recognition tasks.

Introduction

Human Activity Recognition (HAR) is a well-established problem in computer vision with applications spanning surveillance, healthcare monitoring, human-computer interaction, and smart environments. The objective of this project is to build a robust image and video classification system capable of identifying a diverse set of everyday human activities from visual data.

InsightFlow addresses this using a deep convolutional neural network trained via transfer learning on a labelled image dataset spanning 15 activity classes, designed to generalize across both images and video frames. The system reports not just a top-1 prediction but also top-3 candidate activities with associated confidence scores — useful in scenarios where visual ambiguity exists between related activities (e.g., sitting vs. sleeping, or texting vs. using a laptop).

2.1 Objectives

- Design and train a CNN-based classifier for 15 human activity classes.
- Use transfer learning to achieve strong accuracy with limited training data and compute.
- Build a preprocessing and augmentation pipeline appropriate for the target input size ($224 \times 224 \times 3$).
- Evaluate model performance using standard classification metrics.
- Extend image-based inference to video, using frame sampling and prediction averaging.
- Report top-3 predictions with confidence scores for interpretability.

Dataset

The dataset consists of RGB images organized into class-labelled subdirectories, following the structure `dataset/train/<class_name>/*.jpg`. This directory-based layout allows the dataset to be loaded directly using Keras' `image_dataset_from_directory` utility, which infers class labels from folder names and applies a consistent resize and batching pipeline.

3.1 Dataset Source

The dataset used for this project is the **Human Action Recognition (HAR) Dataset**, publicly available on Kaggle:

<https://www.kaggle.com/datasets/meetnagadia/human-action-recognition-har-dataset>

The dataset is contributed by Meet Nagadia on Kaggle and features 15 distinct classes of human activities captured as RGB images, matching the class taxonomy adopted for this project.

3.2 Activity Classes

The dataset spans the following 15 activity classes:

Class	Class	Class
Calling	Eating	Running
Clapping	Fighting	Sitting
Cycling	Hugging	Sleeping
Dancing	Laughing	Texting
Drinking	Listening to Music	Using Laptop

3.3 Train–Validation Split

The dataset is split into training and validation subsets using an 80/20 ratio via Keras' built-in `validation_split` and `subset` parameters, with a fixed random seed (42) to ensure reproducibility across runs. Both subsets are loaded at a fixed batch size of 32 and image size of 224×224 .

3.4 Data Augmentation

A data augmentation pipeline is defined using Keras preprocessing layers: random horizontal flip, random rotation (± 0.15), random zoom (± 0.15), and random contrast adjustment (0.1). This pipeline is designed to improve model robustness to variations in orientation, scale, and lighting conditions typically encountered in real-world activity images.

Methodology

4.1 Preprocessing Pipeline

All input images are resized to a fixed dimension of $224 \times 224 \times 3$ to match the expected input shape of the MobileNetV2 backbone. Normalization is handled internally by Keras' `mobilenet_v2.preprocess_input` function, which is embedded directly within the model graph — this ensures that the same preprocessing is automatically and consistently applied during both training and inference, without requiring a separate manual scaling step.

4.2 Model Architecture

Three candidate architectures were implemented during development: a baseline custom CNN, a MobileNetV2-based transfer learning model, and an EfficientNetB0-based transfer learning model. The final

trained model uses the MobileNetV2 architecture, selected for its strong accuracy-to-efficiency ratio, which is well suited to near real-time inference.

The MobileNetV2-based model is structured as follows:

- Base network: MobileNetV2 pretrained on ImageNet, used as a feature extractor.
- Fine-tuning: the last 30 layers of the base network are unfrozen and fine-tuned; earlier layers remain frozen to retain general low-level visual features.
- Preprocessing layer: mobilenet_v2.preprocess_input applied directly within the model's functional graph.
- Global Average Pooling layer to reduce spatial feature maps to a single feature vector per image.
- Dropout layer (rate = 0.3) for regularization.
- Dense output layer with softmax activation across 15 classes.

4.3 Training Configuration

Parameter	Value
Optimizer	Adam, learning rate = 1e-4
Loss Function	Sparse Categorical Crossentropy
Evaluation Metric	Accuracy
Batch Size	32
Maximum Epochs	20
Input Shape	224 × 224 × 3

4.4 Callbacks

- EarlyStopping — monitors validation loss, stops training if no improvement for 5 consecutive epochs, and restores the best observed weights.
- ModelCheckpoint — saves the model with the highest validation accuracy to best_model.keras during training.
- ReduceLROnPlateau — reduces the learning rate by a factor of 0.2 if validation loss plateaus for 2 consecutive epochs, allowing finer convergence.

Implementation

The complete pipeline was implemented in Python using TensorFlow / Keras for model building and training, OpenCV for video frame extraction, PIL for image handling, and scikit-learn for evaluation metrics. The training and inference pipeline is packaged as a single, sequentially executable notebook designed to run end-to-end in Google Colab, using Google Drive for persistent dataset and model storage across sessions.

5.1 Image Inference Pipeline

For single-image prediction, the input image is converted to RGB, resized to 224×224 , cast to float32, and expanded with a batch dimension before being passed to the trained model. The model outputs a probability distribution across all 15 classes; the class with the highest probability is reported as the top-1 prediction, and the three highest-probability classes are reported as the top-3 predictions with their associated confidence scores (expressed as percentages).

5.2 Video Inference Pipeline

For video prediction, frames are sampled at a fixed interval (every 10th frame) to balance inference cost against representativeness of the clip. Each sampled frame is converted from BGR to RGB, resized to 224×224 , and passed through the model individually. The per-frame prediction probability vectors are then averaged across all sampled frames, and the class with the highest average probability is reported as the final predicted activity, along with the corresponding top-3 predictions and confidence score.

Results and Evaluation

Model performance was evaluated on the held-out validation split (20% of the dataset), which was not used during training. Evaluation was performed with a fresh, non-shuffled validation data loader to ensure correct alignment between predicted and ground-truth labels for metric computation.

6.1 Overall Performance

Metric	Value
Validation Accuracy	93.26%
Validation Loss	0.2853

6.2 Classification Report

A per-class classification report was generated, reporting precision, recall, and F1-score for each of the 15 activity classes. This breakdown highlights classes that are more prone to confusion with visually or contextually similar activities (for example, sitting versus sleeping, or texting versus using a laptop).

6.3 Confusion Matrix

A confusion matrix was plotted to visualize prediction patterns across all 15 classes, highlighting which activity pairs are most frequently confused by the model. This matrix is saved as confusion_matrix.png as part of the evaluation pipeline.

6.4 Training Curves

Training and validation accuracy, as well as training and validation loss, were tracked across all epochs and plotted to assess convergence behaviour and check for overfitting. These curves are saved as training_history.png.

Conclusion

This project successfully implements an end-to-end Human Activity Recognition system capable of classifying 15 everyday human activities from both static images and video clips. By leveraging transfer learning with a MobileNetV2 backbone, the system achieves a practical balance between classification accuracy and computational efficiency, making it suitable for near real-time deployment scenarios. The complete pipeline — from dataset preprocessing through training, evaluation, and inference — was implemented and validated in a single, reproducible notebook environment.

Future Scope

- Expand the dataset with more samples per class and greater diversity in backgrounds, lighting conditions, and camera angles to improve generalization.
- Benchmark the EfficientNetB0 architecture more rigorously against MobileNetV2 to evaluate accuracy versus latency trade-offs for deployment.
- Integrate the defined data augmentation pipeline directly into the training loop to make active use of it during model fitting.
- Support real-time video and webcam-based inference with continuous frame streaming rather than fixed-interval sampling.
- Apply model compression techniques such as quantization or pruning for deployment on edge and mobile devices.
- Extend the system to multi-person activity recognition by combining the classifier with person detection and tracking.
- Incorporate temporal modeling (e.g., LSTM or 3D-CNN architectures over frame sequences) to better capture motion dynamics for activities such as fighting, dancing, and cycling.